Pemrograman berorientasi objek (PBO)

Abstract Class dan Interface



- TIM PBO -

Outline

1. **Review Inheritance**
   extends & @Override
2. **Abstract Class**
   class umum & abstract method
3. **Interface**
   kontrak kemampuan & implements
4. **Multiple Interface**
   satu class, banyak kemampuan
5. **Notasi UML**
   cara menggambar di class diagram
6. **Abstract Class vs Interface**
   perbedaan & kapan dipakai

Capaian Pembelajaran

Setelah mempelajari materi ini, mahasiswa mampu:

- Menjelaskan **konsep abstract class** dan **abstract method**
- Membuat class turunan yang **mengimplementasikan (meng-override) abstract method**
- Menjelaskan **konsep interface** dan membuat **class yang meng-implements interface**
- Menerapkan lebih dari satu interface (multiple interface) pada sebuah class
- Menggambar notasi UML dan memilih kapan memakai **abstract class atau interface**

Ingat Kembali: Inheritance

```
class Hewan {
    void bergerak() {
        System.out.println("Hewan bergerak");
    }
}

class Kucing extends Hewan {
    @Override
    void bergerak() {
        System.out.println("Kucing berjalan");
    }
}
```

`extends`

Kucing mewarisi atribut dan method milik Hewan.

`@Override`

Kucing menulis ulang cara bergerak versinya sendiri.

Tapi...

“Hewan bergerak” itu bergerak bagaimana? Hewan apa? Isi method ini sebenarnya tidak bermakna.

01

Abstract Class



Class umum yang menjadi “aturan” bagi class-class turunannya

Analogi: Semua Hewan Bergerak

T: Apakah semua hewan bisa bergerak?

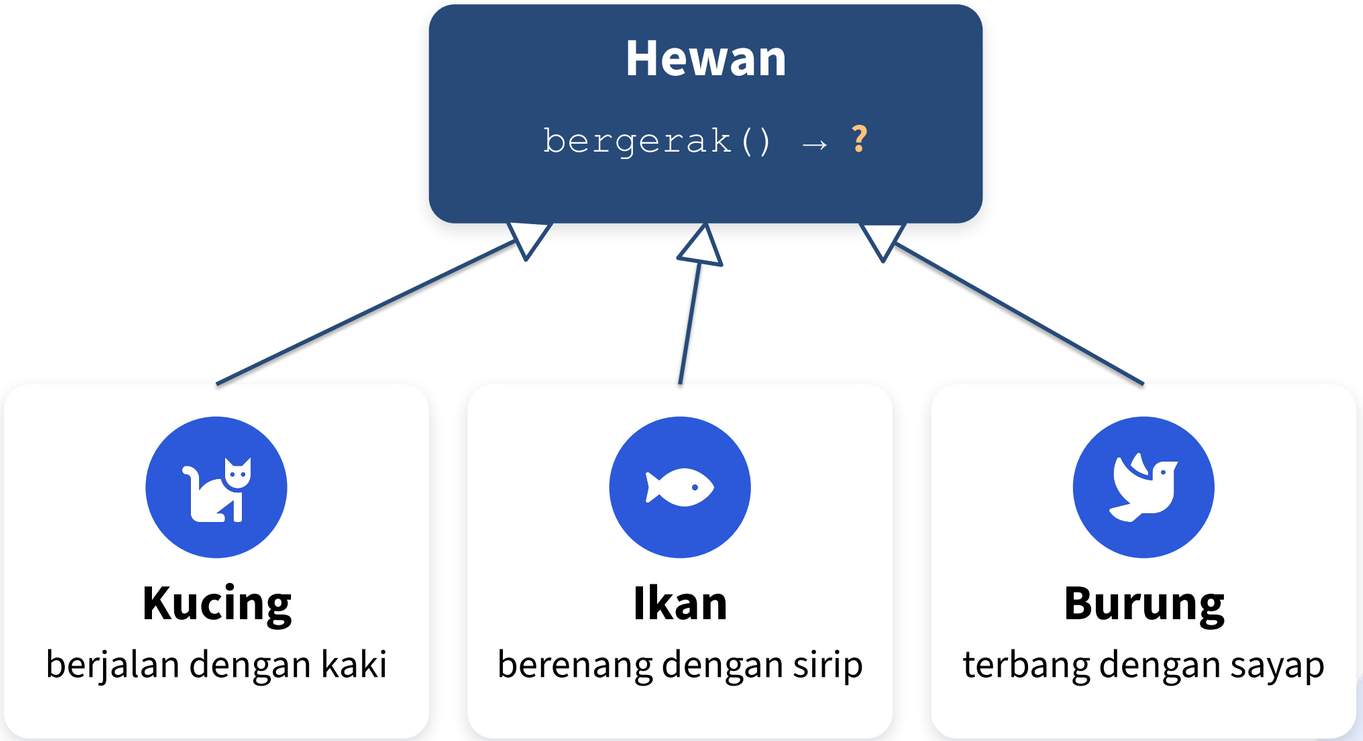
J: Ya, pasti bisa.

T: Bagaimana cara hewan bergerak?

J: Tergantung hewannya apa!

T: Apakah semua hewan bertambah umur?

J: Ya, dan caranya sama untuk semua.



Kesimpulan: Hewan cocok dibuat sebagai abstract class. Semua hewan PASTI bisa bergerak(), tetapi CARA bergeraknya baru ditentukan di class turunan (Kucing, Ikan, Burung).

Apa Itu Abstract Class?

Abstract class adalah class yang **tidak dapat diinstansiasi** (tidak bisa dibuat objeknya), tetapi **dapat di-extend**. Abstract class baru bisa dimanfaatkan melalui class turunannya.

Tidak bisa di-new

`new Hewan(...)` menghasilkan error. Objek dibuat dari class turunannya, misalnya `new Kucing(...)`

Harus di-extend

Menjadi induk (superclass) bagi class-class yang lebih spesifik.

Isinya campuran

Boleh punya atribut, constructor, method biasa, **dan** method abstract.

Analogi: formulir pendaftaran. Kolom yang sama untuk semua orang sudah tercetak, tetapi beberapa kolom WAJIB diisi sendiri oleh setiap pendaftar.

Mengapa Perlu Abstract Class?

1

Induk membuat aturan

Hewan menyatakan: setiap hewan WAJIB punya method `bergerak()`

2

Anak wajib mengisi

Kucing, Ikan, Burung menuliskan cara bergeraknya masing-masing.

3

Pemanggilan seragam

Semua objek pasti bisa dipanggil `h.bergerak()` dengan nama, parameter, dan tipe kembalian yang sama.

Jika class turunan lupa mengisi method abstract, program **gagal dikompilasi**. Kesalahan langsung ketahuan sebelum program dijalankan.

Sintaks Abstract Class

Hewan.java

```java
public abstract class Hewan {                                   // 1
    protected String nama;
    protected int umur;

                                                                // 2
    public Hewan(String nama) {
        this.nama = nama;
    }
                                                                // 3
    public void tambahUmur() {
        umur++;
        System.out.println(nama + " berumur " + umur + " tahun");
    }                                                           // 4

    public abstract void bergerak();
```

1. **abstract class** → class ini tidak bisa di-new, hanya bisa di-extend.
2. **Atribut & constructor** tetap boleh ada, seperti class biasa.
3. **Method konkret** punya isi { } dan langsung diwariskan ke class turunan.
4. **Method abstract** hanya deklarasi: tanpa { }, diakhiri titik koma ;

Abstract Method vs Method Biasa

Method biasa (konkret)

```
public void tambahUmur() {
    umur++;
}
```

- Punya badan method { }
- Bisa langsung dipakai oleh class turunan
- Boleh di-override (tidak wajib)

Method abstract

```
public abstract void bergerak();
```

- Tanpa badan method, diakhiri ;
- Hanya boleh ada di abstract class (atau interface)
- WAJIB di-override oleh class turunan

Aturan: jika sebuah class memiliki minimal satu method abstract, class tersebut WAJIB dideklarasikan sebagai abstract class.

Class Turunan Wajib Meng-override

Kucing.java

```java
public class Kucing extends Hewan {
    public Kucing(String nama) {
        super(nama);
    }

    @Override
    public void bergerak() {
        System.out.println(nama + " berjalan dengan kaki");
    }
}
```

Ikan.java (ringkas)

```java
public class Ikan extends Hewan {
    // constructor sama seperti Kucing
    @Override
    public void bergerak() {
        System.out.println(nama + " berenang dengan sirip");
    }
}
```

`extends Hewan`
Kucing dan Ikan adalah turunan Hewan.

`super(nama)`
Memanggil constructor milik Hewan untuk mengisi atribut nama.

`@Override`
Menandai bahwa bergerak() mengisi method abstract milik induk. Compiler ikut mengecek nama method.

Strukturnya sama; yang berbeda hanya isi method bergerak().

Menjalankan Program

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Hewan[] daftar = { new Kucing("Tom"), new Ikan("Nemo") };

        for (Hewan h : daftar) {
            h.bergerak();
            h.tambahUmur();
        }
    }
}
```

Output

```
$ java Main
Tom berjalan dengan kaki
Tom berumur 1 tahun
Nemo berenang dengan sirip
Nemo berumur 1 tahun
```

Variabel bertipe **`Hewan`** boleh menampung objek **`Kucing`** maupun **`Ikan`**.

`bergerak()` yang dijalankan mengikuti objek aslinya. Konsep ini dibahas lebih dalam di materi **Polimorfisme**.

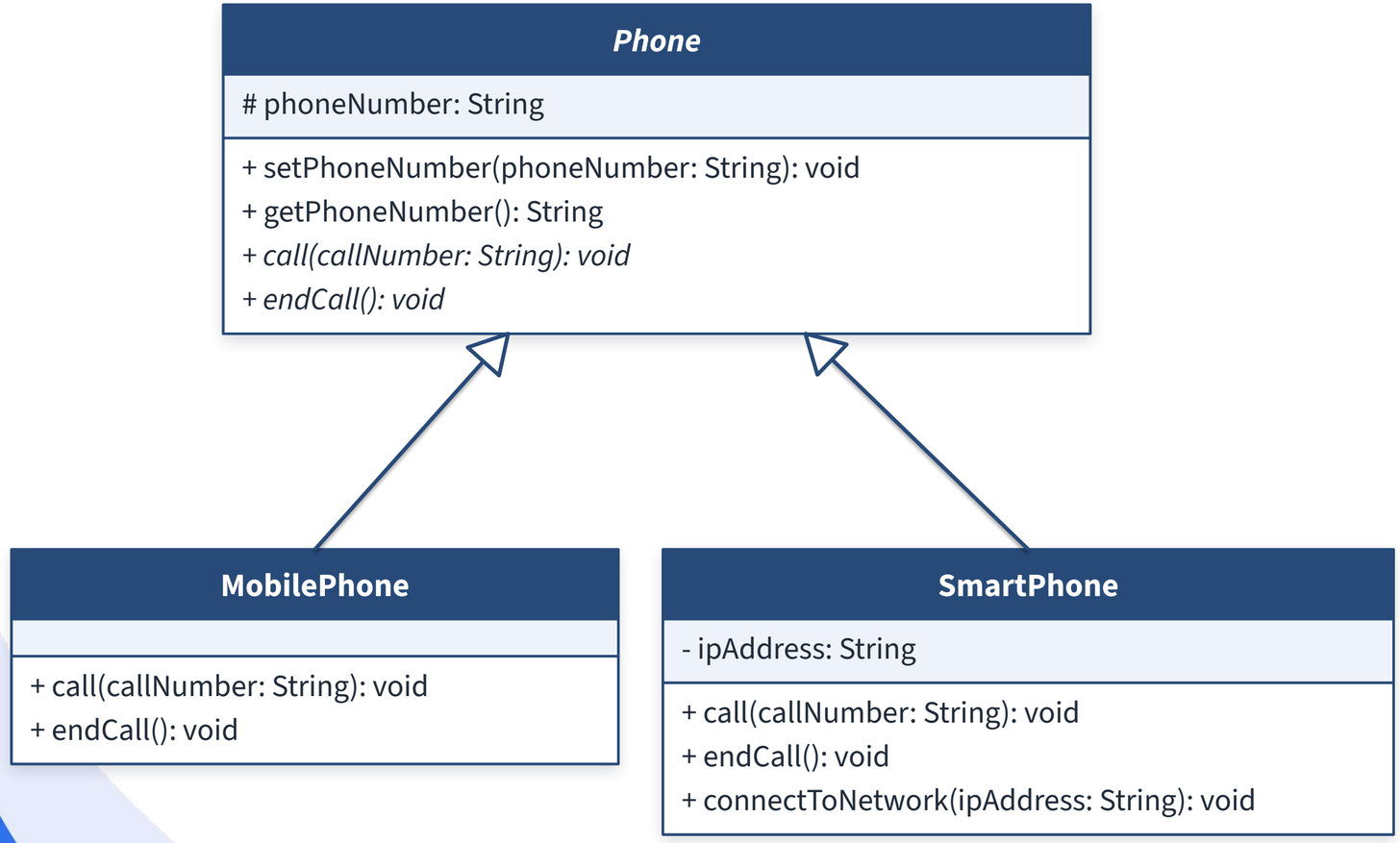
Error yang Sering Muncul

Kode yang salah	Pesan error dari compiler	Perbaikan
`Hewan h = new Hewan("Tom");`	✗ `error: Hewan is abstract; cannot be instantiated`	✓ Buat objek dari class turunan: **`new Kucing("Tom")`**

`class Burung extends Hewan {`
`  // lupa bergerak()`
`}`
	✗ `error: Burung is not abstract and does not override abstract method bergerak() in Hewan`	✓ Tambahkan method **`bergerak()`** dengan @Override di Burung

`abstract void bergerak() {`
`  ...`
`}`
	✗ `error: abstract methods cannot have a body`	✓ Hapus { } dan akhiri dengan titik koma **;**

Notasi UML Abstract Class



Nama class abstract ditulis miring (italic). Boleh ditambah «abstract» di atas nama.

Method abstract juga ditulis miring.



Garis penuh + segitiga kosong = generalisasi (extends), mengarah ke class induk.

Studi Kasus: Phone

Phone.java (abstract class)

```java
public abstract class Phone {
    protected String phoneNumber;

    public void setPhoneNumber(String phoneNumber) {
        this.phoneNumber = phoneNumber;
    }

    public String getPhoneNumber() {
        return phoneNumber;
    }

    public abstract void call(String callNumber);
    public abstract void endCall();
}
```

MobilePhone.java (class turunan)

```java
public class MobilePhone extends Phone {
    @Override
    public void call(String callNumber) {
        System.out.println(
            "MobilePhone menelepon ke: " + callNumber);
    }

    @Override
    public void endCall() {
        System.out.println(
            "MobilePhone mengakhiri panggilan");
    }
}
```

`MobilePhone` mewarisi setPhoneNumber() dan getPhoneNumber() apa adanya, dan **wajib** mengisi call() serta endCall().

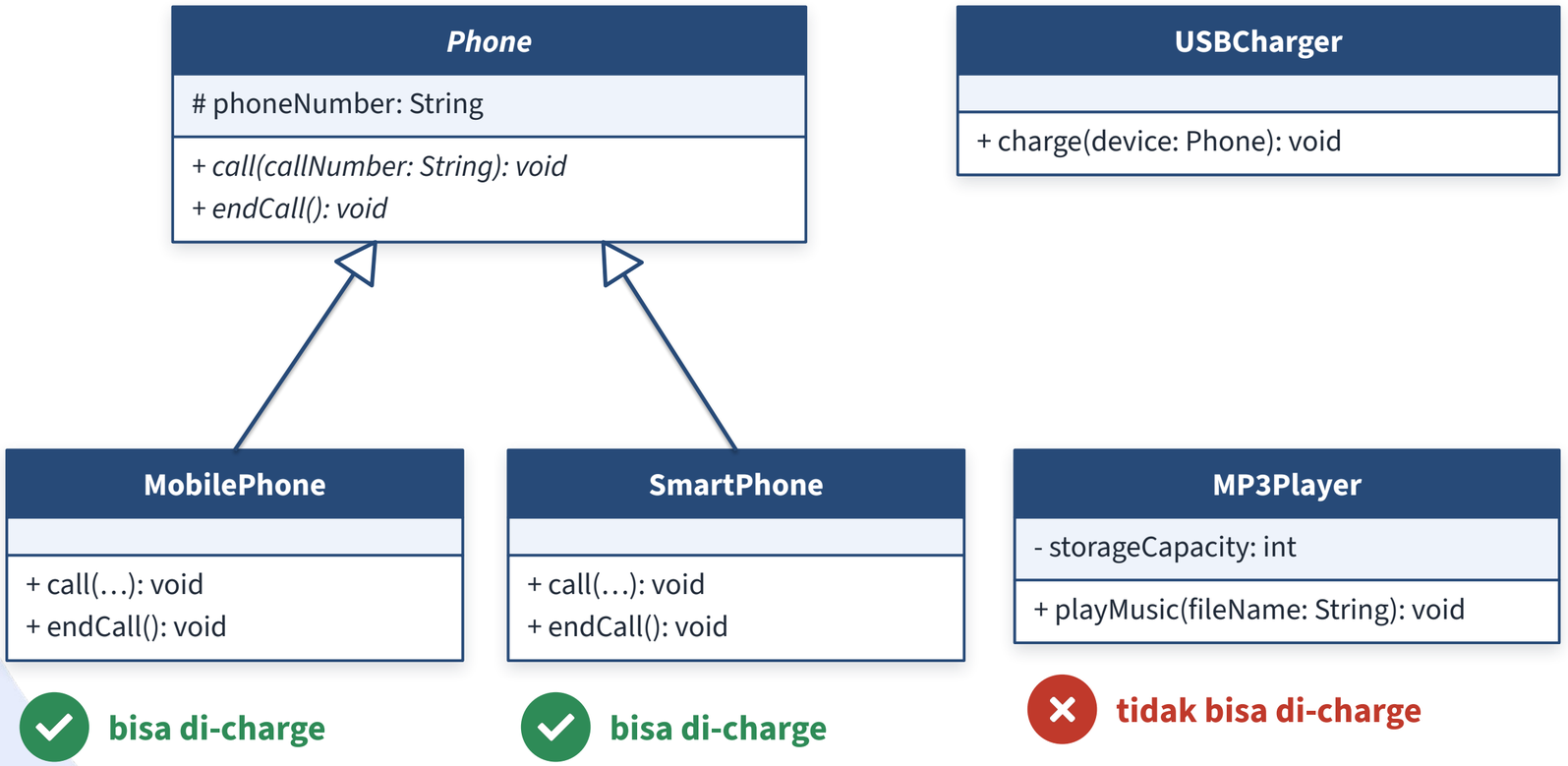
02

Interface

Kontrak kemampuan yang bisa dipakai oleh class mana pun, apa pun induknya



Masalah: Siapa yang Bisa Di-charge?



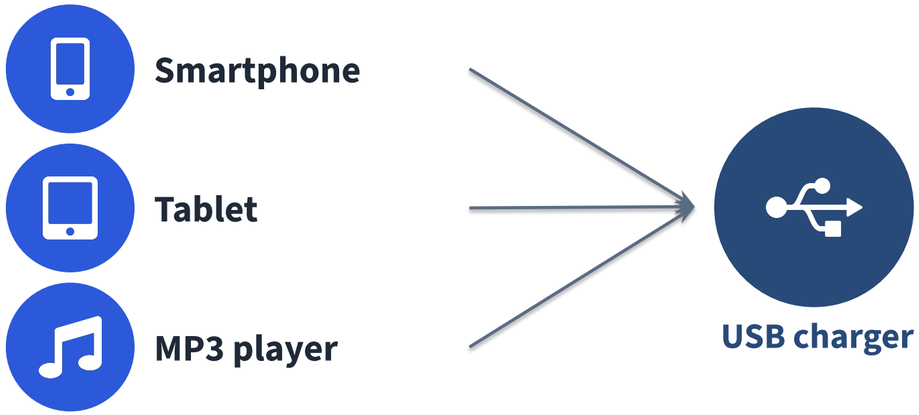
- `charge()` hanya menerima objek bertipe **`Phone`**.
- MobilePhone & SmartPhone turunan Phone → bisa.
- MP3Player bukan turunan Phone → tidak bisa.
- **MP3Player extends Phone?** Tidak masuk akal: MP3Player bukan telepon dan tidak bisa call()!

Bagaimana agar semua perangkat bisa di-charge tanpa memaksa mereka satu keturunan? → **Gunakan interface**

Apa Itu Interface?

Interface adalah kontrak berisi daftar kemampuan (method) yang **wajib dimiliki** oleh class yang meng-implements-nya, tanpa memedulikan hierarki pewarisan class tersebut.

Analogi: port USB



Berbeda jenis, tetapi sama-sama punya port USB → bisa memakai charger yang sama.

Abstract class → hubungan ADALAH (is-a)

Contoh: *SmartPhone adalah Phone*

Interface → hubungan BISA (can-do)

Contoh: *MP3Player bisa di-charge (ChargeAble)*

Tips: nama interface sering berakhiran **-able**: ChargeAble, Comparable, Runnable.

Membuat & Meng-implements Interface

ChargeAble.java

```java
public interface ChargeAble {
    void doCharge();
}
```

MP3Player.java

```java
public class MP3Player implements ChargeAble {
    private int storageCapacity;

    public void playMusic(String fileName) {
        System.out.println("Memutar lagu: " + fileName);
    }

    @Override
    public void doCharge() {
        System.out.println("MP3Player sedang di-charge");
    }
}
```

1. **interface** → mendefinisikan kontrak, bukan class biasa.
2. **Method cukup deklarasi**; otomatis public abstract.
3. **implements** → MP3Player menyetujui kontrak ChargeAble.
4. **Wajib mengisi doCharge()** dan aksesnya harus public.

Aturan Isi Interface

1 Method otomatis public abstract

```
void doCharge();
```

sama artinya dengan public abstract void doCharge();

2 Atribut otomatis konstanta

```
int MAX_VOLT = 5;
```

otomatis public static final, nilainya tidak bisa diubah.

3 Tidak bisa di-new

```
new ChargeAble();
```

error: ChargeAble is abstract; cannot be instantiated.
Interface juga tidak punya constructor.

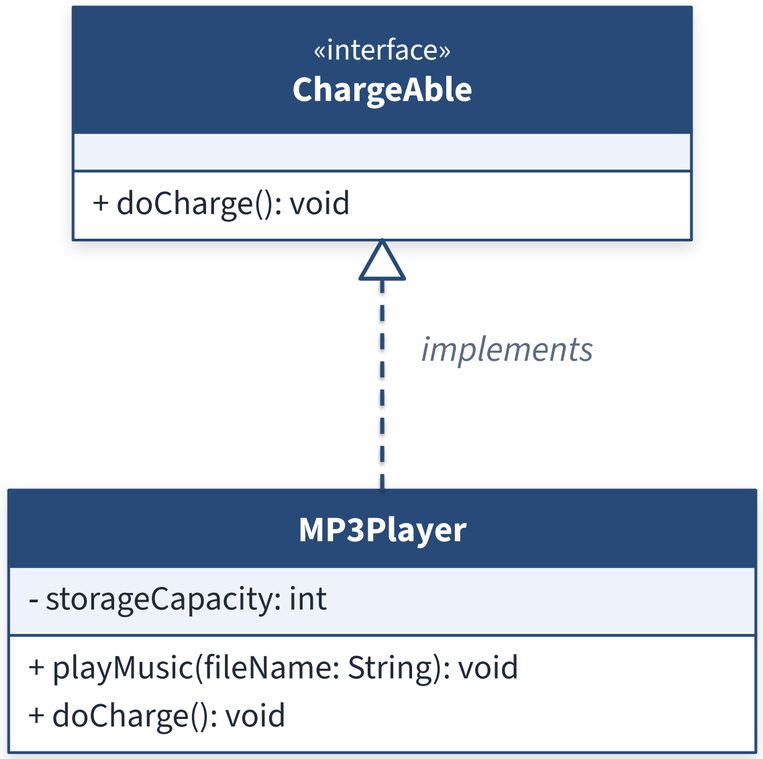
4 Implementasi harus public

```
void doCharge() { … }
```

tanpa public → error: attempting to assign weaker access privileges; was public

Info Java 8 ke atas: interface juga boleh memiliki default method dan static method yang punya isi. Pada pertemuan ini kita fokus pada method abstract.

Notasi UML Interface



«interface» ditulis di atas nama interface.

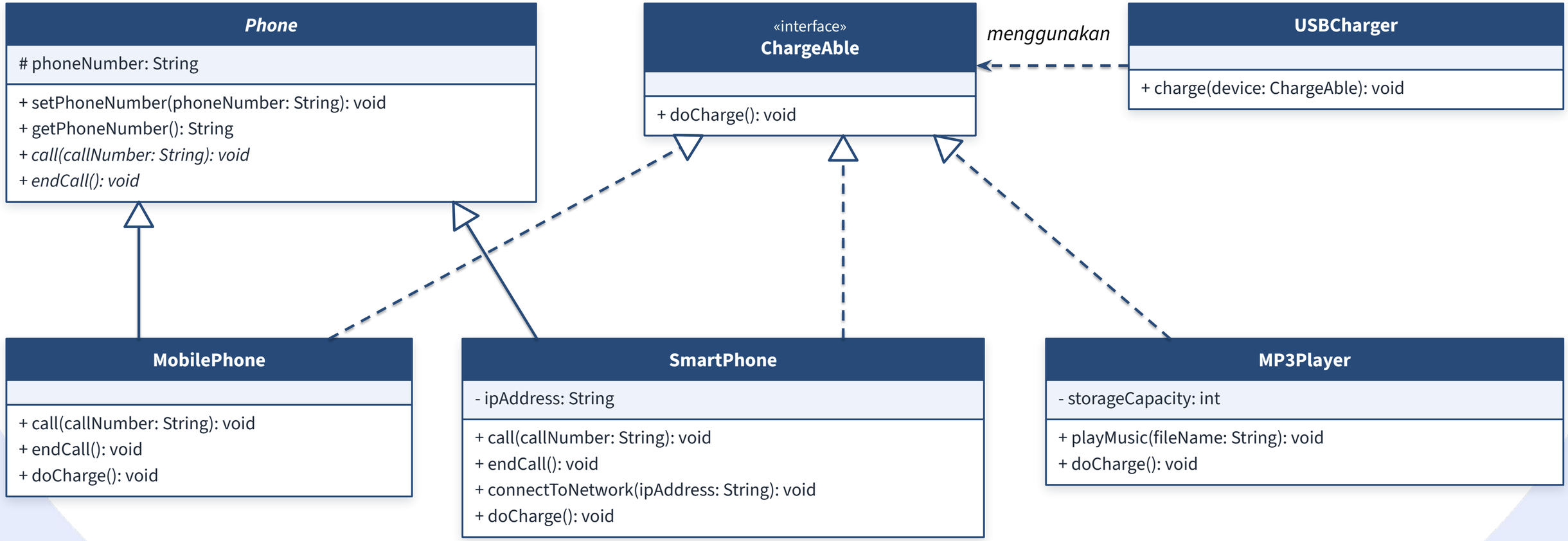


Garis putus-putus + segitiga kosong = realisasi (implements).



Bandingkan: garis penuh + segitiga kosong = generalisasi (extends).

Solusi: Menerapkan Interface



Hasilnya: semua yang meng-implements ChargeAble bisa di-charge oleh USBCharger, tanpa harus satu keturunan.

Solusi: Kode Program

USBCharger.java

```java
public class USBCharger {
    public void charge(ChargeAble device) {
        System.out.print("[USBCharger] ");
        device.doCharge();
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        USBCharger charger = new USBCharger();

        charger.charge(new MobilePhone());
        charger.charge(new SmartPhone());
        charger.charge(new MP3Player());
    }
}
```

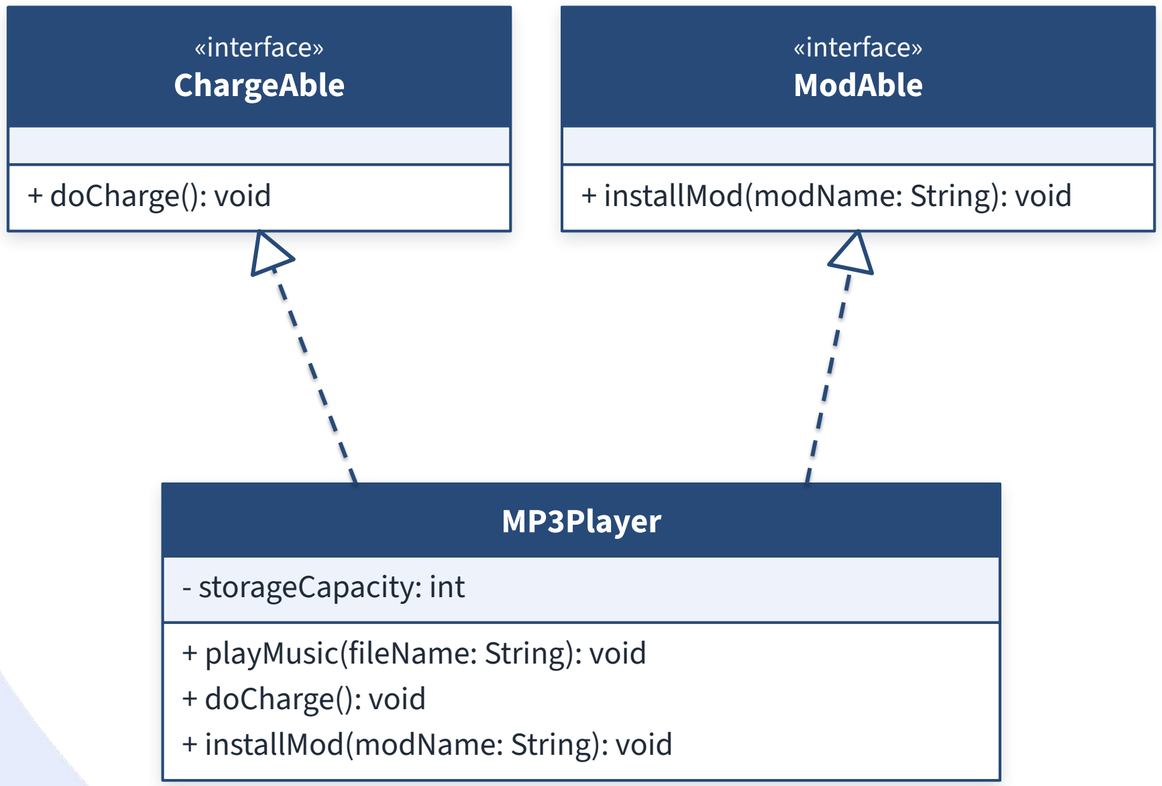
Output

```
$ java Main
[USBCharger] MobilePhone sedang di-charge
[USBCharger] SmartPhone sedang fast charging
[USBCharger] MP3Player sedang di-charge
```

Parameter bertipe interface **`(ChargeAble device)`** artinya: objek apa pun yang meng-implements ChargeAble boleh masuk.

USBCharger tidak perlu tahu jenis perangkatnya. Cukup tahu bahwa perangkat itu **bisa doCharge()**.

Multiple Interface



Satu class boleh meng-implements **lebih dari satu** interface, dipisah koma.

```
class MP3Player
    implements ChargeAble, ModAble
```

Boleh digabung dengan extends: **extends dulu, baru implements**.

```
class SmartPhone extends Phone
    implements ChargeAble, ModAble
```

Tetapi extends hanya boleh **SATU** class. `class X extends A, B` → error.

Contoh Multiple Interface

MP3Player.java

```java
public class MP3Player implements ChargeAble, ModAble {
    private int storageCapacity;

    public void playMusic(String fileName) {
        System.out.println("Memutar lagu: " + fileName);
    }

    @Override
    public void doCharge() {
        System.out.println("MP3Player sedang di-charge");
    }

    @Override
    public void installMod(String modName) {
        System.out.println("MP3Player memasang mod: " + modName);
    }
}
```

ModAble.java

```java
public interface ModAble {
    void installMod(String modName);
}
```

Isi method main()

```java
MP3Player mp3 = new MP3Player();
mp3.playMusic("Bengawan Solo.mp3");
mp3.doCharge();
mp3.installMod("Equalizer Pro");
```

Output

```
$ java Main
Memutar lagu: Bengawan Solo.mp3
MP3Player sedang di-charge
MP3Player memasang mod: Equalizer Pro
```

03

Abstract Class vs Interface



Memilih alat yang tepat untuk kebutuhan yang tepat

Perbandingan

Aspek	Abstract Class	Interface
Kata kunci	abstract class … / extends	interface … / implements
Bisa di-new?	Tidak	Tidak
Isi method	Abstract dan konkret (punya isi)	Abstract
Atribut	Bebas (private, protected, dll.)	Hanya konstanta public static final
Constructor	Boleh punya	Tidak punya
Jumlah per class	Hanya extends 1 class	Boleh implements banyak
Hubungan	“adalah” (is-a): Kucing adalah Hewan	“bisa” (can-do): MP3Player bisa di-charge
Notasi UML	Nama miring, panah garis penuh	«interface», panah garis putus-putus

Kapan Menggunakan?

Gunakan abstract class jika…

- Class-class berhubungan erat (“adalah”), mis. MobilePhone dan SmartPhone adalah Phone
- Ingin berbagi kode yang sama (atribut, method konkret) ke class turunan
- Butuh atribut biasa (bukan konstanta) atau constructor

Gunakan interface jika…

- Class-class tidak satu hierarki, tetapi butuh kemampuan yang sama (MobilePhone dan MP3Player bisa di-charge)
- Hanya ingin menentukan APA yang harus bisa dilakukan, bukan BAGAIMANA
- Satu class butuh beberapa kemampuan sekaligus (multiple interface)

Tips cepat: “X adalah Y?” → abstract class. “X bisa Z?” → interface. Keduanya boleh dipakai bersamaan.

Rangkuman

1	**Abstract class**	class umum yang tidak bisa di-new; dipakai lewat extends.
2	**Abstract method**	method tanpa isi; class turunan wajib meng-override.
3	**Interface**	kontrak kemampuan; dipakai lewat implements.
4	**Aturan pewarisan**	satu class hanya extends 1 class, tetapi boleh implements banyak interface.
5	**UML**	abstract ditulis miring; interface diberi «interface» dan panah garis putus-putus.

Latihan

1. Class yang tidak dapat diinstansiasi namun dapat di-extend disebut ….
2. Method yang hanya dideklarasikan tanpa implementasi disebut ….
3. Keyword untuk menggunakan interface pada sebuah class adalah ….
4. Apa yang terjadi jika class turunan tidak meng-override semua abstract method induknya?
5. Mengapa MP3Player lebih tepat meng-implements ChargeAble daripada meng-extends Phone?
6. Sebutkan 3 perbedaan abstract class dan interface!

TERIMA KASIH

